LAB-01 Solutions

Object Oriented Programming - University of Fallujah, College of Information Technology

Name: _____ Section: _____

Task 01

Code

```
#include <iostream>
using namespace std;

// takes an array and its size and returns a new array of double size
// first half = original elements, second half = 2 * original elements
int* resizeArray(int inArray[], const int size)
{
    int* newArray = new int[size * 2];

    for (int i = 0; i < size; i++)
    {
        newArray[i] = inArray[i];
        newArray[i + size] = inArray[i] * 2;
    }

    return newArray;
}

int main()
{
    int size;

    cout << "Enter size of the array: ";
    cin >> size;

    if (size <= 0)
    {
        cout << "Size must be greater than zero." << endl;
        return 0;
    }

    int* arr = new int[size];

    cout << "Enter elements of array: ";
    for (int i = 0; i < size; i++)
    {
        cin >> arr[i];
    }

    int* resized = resizeArray(arr, size);

    cout << "Resized array: ";
    for (int i = 0; i < size * 2; i++)
    {
        cout << resized[i];
        if (i < size * 2 - 1)
        {
            cout << ", ";
        }
    }
    cout << endl;

    // free the memory
    delete[] arr;
    delete[] resized;
    arr = nullptr;
    resized = nullptr;

    return 0;
}
```

Output

```
Enter size of the array: 4
```

```
Enter elements of array: 1 2 3 4
Resized array: 1, 2, 3, 4, 2, 4, 6, 8
```

Task 02

Code

```
#include <iostream>
using namespace std;

// returns true if all entries outside the main diagonal are zero
bool isDiagonal(int** matrix, const int n)
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            // i != j means the element is not on the main diagonal
            if (i != j && matrix[i][j] != 0)
            {
                return false;
            }
        }
    }
    return true;
}

int main()
{
    int n;

    cout << "Enter size of the square matrix: ";
    cin >> n;

    if (n <= 0)
    {
        cout << "Size must be greater than zero." << endl;
        return 0;
    }

    // create the 2-D array dynamically
    int** matrix = new int*[n];
    for (int i = 0; i < n; i++)
    {
        matrix[i] = new int[n];
    }

    // take the values from the user
    cout << "Enter the elements of the matrix:" << endl;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << "Element [" << i << "][" << j << "]: ";
            cin >> matrix[i][j];
        }
    }

    // display the matrix
    cout << endl << "Matrix:" << endl;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << matrix[i][j] << "\t";
        }
        cout << endl;
    }

    if (isDiagonal(matrix, n))
    {
        cout << endl << "The matrix is a diagonal matrix." << endl;
    }
    else
    {
        cout << endl << "The matrix is NOT a diagonal matrix." << endl;
```

```
    }

    // free the memory
    for (int i = 0; i < n; i++)
    {
        delete[] matrix[i];
    }
    delete[] matrix;
    matrix = nullptr;

    return 0;
}
```

Output (run 1)

```
Enter size of the square matrix: 3
Enter the elements of the matrix:
Element [0][0]: 1
Element [0][1]: 0
Element [0][2]: 0
Element [1][0]: 0
Element [1][1]: 4
Element [1][2]: 0
Element [2][0]: 0
Element [2][1]: 0
Element [2][2]: -2

Matrix:
1       0       0
0       4       0
0       0       -2

The matrix is a diagonal matrix.
```

Output (run 2)

```
Enter size of the square matrix: 2
Enter the elements of the matrix:
Element [0][0]: 1
Element [0][1]: 5
Element [1][0]: 0
Element [1][1]: 3

Matrix:
1       5
0       3

The matrix is NOT a diagonal matrix.
```

Task 03

Code

```
#include <iostream>
using namespace std;

// returns the length of a c-string
int stringLength(const char* str)
{
    int len = 0;
    while (str[len] != '\0')
    {
        len++;
    }
    return len;
}

// returns a new string that has the first 'count' characters of target
char* left(const char* target, const int count)
{
    char* result = new char[count + 1];

    for (int i = 0; i < count; i++)
    {
        result[i] = target[i];
    }
    result[count] = '\0';

    return result;
}

// de-allocates the memory pointed by p
void Destructor(char* p)
{
    delete[] p;
}

int main()
{
    char text[30];
    int count;

    cout << "Enter a string: ";
    cin.getline(text, 30);

    // if the user typed more than 29 characters, getline stores the first 29
    // and sets the fail flag, so clear it and throw away the extra characters
    if (cin.fail())
    {
        cin.clear();
        cin.ignore(1000, '\n');
    }

    cout << "Enter count: ";
    cin >> count;

    int length = stringLength(text);

    // check if the count is in range
    if (count < 0 || count > length)
    {
        cout << "Out of range! The string has only " << length << " characters." << endl;
        return 0;
    }

    char* sub = left(text, count);
    cout << "Left substring: " << sub << endl;

    Destructor(sub);
    sub = nullptr;

    return 0;
```

```
}
```

Output (run 1)

```
Enter a string: Equal matrices have equal dimensions
Enter count: 5
Left substring: Equal
```

Output (run 2)

```
Enter a string: Equal matrices
Enter count: 20
Out of range! The string has only 14 characters.
```

Task 04

Code

```
#include <iostream>
using namespace std;

// returns the smallest mark
int findMin(int inArray[], const int size)
{
    int min = inArray[0];
    for (int i = 1; i < size; i++)
    {
        if (inArray[i] < min)
        {
            min = inArray[i];
        }
    }
    return min;
}

// returns the largest mark
int findMax(int inArray[], const int size)
{
    int max = inArray[0];
    for (int i = 1; i < size; i++)
    {
        if (inArray[i] > max)
        {
            max = inArray[i];
        }
    }
    return max;
}

// returns the average of the marks
int computeAverage(int inArray[], const int size)
{
    int sum = 0;
    for (int i = 0; i < size; i++)
    {
        sum += inArray[i];
    }
    return sum / size;
}

// returns the median of the marks
int computeMedian(int inArray[], const int size)
{
    // make a copy so the original order of marks is not changed
    int* temp = new int[size];
    for (int i = 0; i < size; i++)
    {
        temp[i] = inArray[i];
    }

    // sort the copy in ascending order (bubble sort)
    for (int i = 0; i < size - 1; i++)
    {
        for (int j = 0; j < size - i - 1; j++)
        {
            if (temp[j] > temp[j + 1])
            {
                int swap = temp[j];
                temp[j] = temp[j + 1];
                temp[j + 1] = swap;
            }
        }
    }

    int median;
    if (size % 2 == 0)
    {
```

```
        // even count: average of the two middle values
        median = (temp[size / 2 - 1] + temp[size / 2]) / 2;
    }
    else
    {
        // odd count: the middle value
        median = temp[size / 2];
    }

    delete[] temp;
    return median;
}

int main()
{
    int students;

    cout << "Enter the number of students enrolled in OOP: ";
    cin >> students;

    if (students <= 0)
    {
        cout << "Number of students must be greater than zero." << endl;
        return 0;
    }

    int* marks = new int[students];

    for (int i = 0; i < students; i++)
    {
        cout << "Enter marks of student " << i + 1 << ": ";
        cin >> marks[i];

        // do not accept negative marks
        while (marks[i] < 0)
        {
            cout << "Marks cannot be negative. Enter again: ";
            cin >> marks[i];
        }
    }

    cout << endl;
    cout << "Maximum marks : " << findMax(marks, students) << endl;
    cout << "Minimum marks : " << findMin(marks, students) << endl;
    cout << "Average marks : " << computeAverage(marks, students) << endl;
    cout << "Median marks  : " << computeMedian(marks, students) << endl;

    delete[] marks;
    marks = nullptr;

    return 0;
}
```

Output (run 1)

```
Enter the number of students enrolled in OOP: 5
Enter marks of student 1: 80
Enter marks of student 2: -5
Marks cannot be negative. Enter again: 90
Enter marks of student 3: 60
Enter marks of student 4: 75
Enter marks of student 5: 100

Maximum marks : 100
Minimum marks : 60
Average marks : 81
Median marks  : 80
```

Output (run 2)

```
Enter the number of students enrolled in OOP: 4
Enter marks of student 1: 50
Enter marks of student 2: 70
```

```
Enter marks of student 3: 90
Enter marks of student 4: 60

Maximum marks : 90
Minimum marks : 50
Average marks : 67
Median marks  : 65
```